Captura y Almacenamiento de Datos Masivos

Tema 3. Modelos no relacionales

Índice

Esquema

Ideas clave

3.1. Introducción y objetivos

3.2. Modelos no-relacionales

3.3. Modelo documental. MongoDB

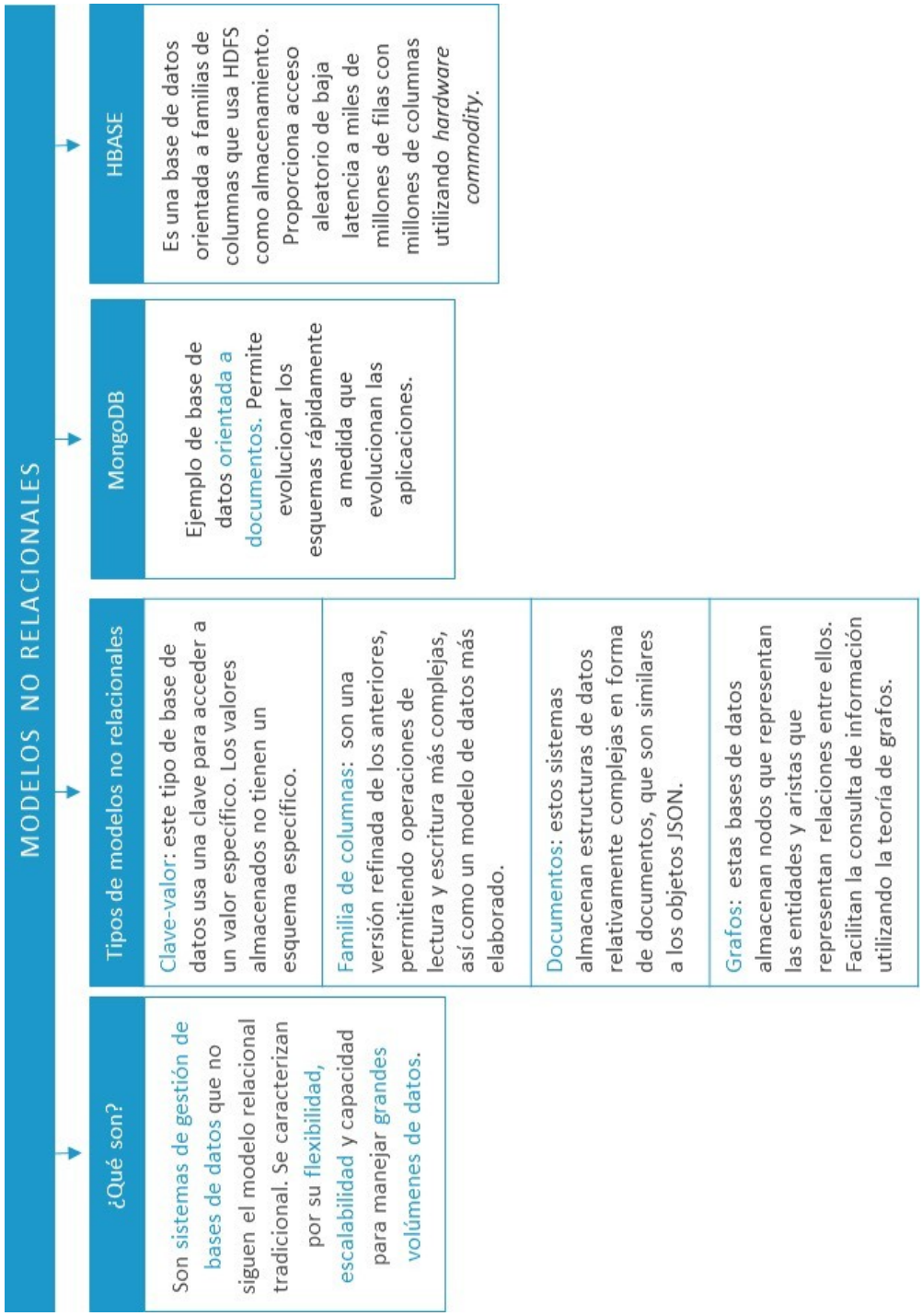
3.4. Modelo columnar. Hbase

3.5. Casos de uso

A fondo

Documentación oficial de MongoDB

Documentación oficial de Apache HBASE



© Universidad Internacional de La Rioja (UNIR)

3.1. Introducción y objetivos

El objetivo principal es comprender las características, ventajas y desventajas de las bases de datos no relacionales, llamadas también bases de datos NoSQL o Not Only SQL. Revisaremos los **distintos tipos de bases de datos NoSQL,** incluyendo las orientadas a clave-valor, las orientadas a familias de columnas, las orientadas a documentos y las orientadas a grafos.

Conocer los diferentes tipos de bases de datos NoSQL y sus características principales nos permitirá **diseñar sistemas de almacenamiento y captura de datos** adaptados a casos de uso concretos, donde el volumen de datos no haga posible el uso de una base de datos relacional.

Veremos los conceptos clave que subyacen a estos sistemas, como la escalabilidad horizontal, la consistencia eventual y la tolerancia a fallos. Veremos, en este contexto, el **teorema CAP,** que describe las limitaciones inherentes a cualquier sistema distribuido en términos de consistencia, disponibilidad y tolerancia al particionado del sistema.

Para ilustrar los conceptos prácticos, se examinarán los casos concretos de dos bases de datos no relacionales populares en la actualidad, MongoDB y HBase. Veremos cómo estas bases de datos ayudan a realizar consultas eficientes y a optimizar el rendimiento de las aplicaciones.

Finalizaremos presentando algunos casos de uso de estas bases de datos en el mundo real.

© Universidad Internacional de La Rioja (UNIR)

3.2. Modelos no-relacionales

En el panorama actual de la tecnología, el manejo de grandes volúmenes de datos se ha convertido en un desafío importante. Las bases de datos relacionales a menudo se quedan cortas para satisfacer las demandas de escalabilidad, flexibilidad y rendimiento que exige el tratamiento de grandes volúmenes de datos. Es en este contexto donde surgen las **bases de datos no relacionales como una alternativa para superar estos retos.**

El acrónimo NoSQL fue utilizado por primera vez en 1998 por Carlo Strozzi para nombrar a una base de datos relacional ligera que no utilizaba SQL. Pero no fue hasta 2009 que este término volvió a surgir, cuando Eric Evans y Johan Oskarsson lo utilizaron para describir las **bases de datos que no utilizaban modelos relacionales.** En este contexto, el término NoSQL puede significar «No usa SQL», pero cada vez es más común que se interprete como «No solo usa SQL», para enfatizar que algunos sistemas NoSQL pueden admitir lenguajes de consulta similares a SQL.

Son varias las razones por las que estas bases de datos han ido surgiendo, la primera sería la **necesidad de una escalabilidad horizontal.** Mientras los RDBMS no están diseñados para este propósito, las bases de datos NoSQL sí tienen de base en su implementación la capacidad de escalar fácilmente **añadiendo más nodos nuevos al sistema.** Esto es importante desde un punto de vista no solo técnico, sino además desde un punto de vista de eficiencia económica. Los sistemas pueden ser escalados mediante inversiones pequeñas y paulatinas.

Otro factor que ha impulsado la creación de las bases de datos NoSQL es la **necesidad de flexibilidad a la hora de evolucionar la estructura de los datos almacenados.** Como vimos, los RDBMS requieren de un esquema fijo predefinido antes de almacenar los datos. Esto hace que sea complejo almacenar datos con

© Universidad Internacional de La Rioja (UNIR)

estructuras variables no conocidas de antemano, así como de trabajar con datos que evolucionan rápidamente. La modificación de un esquema relacional lleva siempre implícito un proceso de migración de los datos del esquema anterior al nuevo.

Por último, surge la **necesidad de acceso a grandes volúmenes de datos** a un alto rendimiento. Muchas de las bases de datos NoSQL están optimizadas para lecturas y escrituras a mayor velocidad que sus contrapartes relacionales, aún a costa de algunas de las garantías que estas ofrecen. En otras ocasiones, el rendimiento se logra mediante la simplificación de los modelos hasta el punto de estar completamente desnormalizados.

Las distintas bases de datos NoSQL han sido organizadas en **cuatro grandes grupos** según la forma en que estructuran los datos. Esta estructuración es, en general, más simple que la del modelo relacional y los tipos son los siguientes:

- Bases de datos orientadas a clave-valor.
- Bases de datos orientadas a columnas o familias de columnas.
- Bases de datos orientadas a documentos.
- Bases de datos orientadas a grafos.

Bases de datos orientadas a clave-valor

Las bases de datos clave-valor son las **más simples** dentro de las NoSQL. Su estructura se **asemeja a la de un diccionario,** donde cada pieza de información se compone de una clave única y un valor asociado. Esta simplicidad las convierte en una opción atractiva para diversas aplicaciones, especialmente, aquellas que requieren un acceso rápido a la información.

© Universidad Internacional de La Rioja (UNIR)

Estas bases de datos no imponen un esquema predefinido ni a la clave ni al valor almacenándolos en la mayoría de los casos, simplemente, como conjuntos de bytes. Esto significa que ambos pueden adoptar dif**erentes formatos, desde simples cadenas de texto hasta datos más complejos.** Esta flexibilidad en los datos las hace ideales para almacenar información no estructurada, como audio, vídeo, fotos o textos en lenguaje natural que no serían fácilmente almacenables y explotables de otra manera.

CLAVE	VALOR
"luis"	"gomez lopez"
3212544	"{\"name\": \"luis\", \"apellidos\": \"gomez lopez\"}
"primos"	[2, 3, 5, 7, 11, 13]
"Bladerunner"	0010101 010101 01010 010101 010101 010101 ....
1.0	"La unidad como un float"

Tabla 1. Ejemplo de datos clave-valor. Fuente: elaboración propia.

Otra ventaja radica en su **eficiencia,** dado que la búsqueda de un valor se realiza directamente a través de su clave única, proceso que se puede optimizar de múltiples formas y que garantiza un acceso rápido. Además, su arquitectura suele ser **muy simple,** lo que facilita que sean escalables horizontalmente y permite agregar nodos al sistema sin comprometer el rendimiento.

Un ejemplo de uso de estas bases de datos es servir de **sistemas de caché para otros almacenamientos más complejos.** Un ejemplo popular de este uso es Memcached, una base de datos NoSQL de código abierto utilizada para acelerar aplicaciones web. Memcached almacena datos en memoria, lo que permite un acceso ultra rápido a la información frecuentemente solicitada, reduce la carga en otros sistemas de almacenamiento de las aplicaciones y acelera el rendimiento de estas.

© Universidad Internacional de La Rioja (UNIR)

Bases de datos orientadas a columnas o familias de columnas

Las bases de datos orientadas a columnas, a diferencia de las bases de datos relacionales que almacenan los datos en filas, **organizan la información en columnas.** Este enfoque ofrece ventajas significativas cuando se trata de manejar grandes volúmenes de datos con estructuras diferentes y realizar consultas analíticas complejas.

En una base de datos orientada a columnas, los datos de una misma columna se almacenan juntos en disco, lo que permite una **lectura eficiente,** ya que solo se necesita acceder a las columnas relevantes para una consulta específica. Al contrario, en una base de datos relacional al estar orientada a filas, la lectura de una fila completa es necesaria, incluso si solo se requieren unos pocos valores de esta.

Esta **mejora en la lectura de columnas individuales se traduce en un mejor rendimiento** para las consultas analíticas, que a menudo implican el análisis de grandes conjuntos de datos y la agregación de información de múltiples columnas de forma individual. Además, las bases de datos orientadas a columnas suelen implementar técnicas de compresión avanzadas, lo que reduce el espacio de almacenamiento y mejora aún más la eficiencia de las consultas.

Dentro de las bases de datos columnares hay una versión más especializada llamadas bases de datos orientadas a familias de columnas. Estas son una variante que introduce el concepto de «familias de columnas», que **permiten mejorar aún más la organización y el rendimiento.** En este enfoque, las columnas se agrupan en familias, que actúan como contenedores para columnas relacionadas. Cada familia de columnas se almacena y gestiona como una unidad independiente, lo que permite un acceso más eficiente a los datos y una mayor flexibilidad en el diseño del esquema.

© Universidad Internacional de La Rioja (UNIR)

CLAVE	FAMILIA-1	FAMILIA-2
1125	nombre:juan, edad:25	tipo:alumno, asignatura1:rdbms, asignatura2:nosql
1254	nombre:luis, edad:49, contratado:si	tipo:profesor, asignatura:nosql, catedra:datos
2255	nombre:maria, edad:24	tipo:alumno, asignatura1:nosql, matricula:si
25488	nombre:ana, edad:35, contratado:no	tipo:colaborador, asignatura1:nosql, asignatura2:rdbms, catedra:datos

Tabla 2. Ejemplo de datos en familias de columnas. Fuente: elaboración propia.

Un ejemplo de este último tipo es Apache HBase, un sistema de almacenamiento distribuido de código abierto que forma parte del ecosistema Hadoop. HBase se utiliza para **almacenar grandes cantidades de datos no estructurados o semiestructurados** y es ideal para aplicaciones que requieren un alto rendimiento de lectura y escritura.

Bases de datos orientadas a documentos

Las bases de datos orientadas a documentos representan un tipo de base de datos NoSQL **diseñadas para el almacenamiento, recuperación y gestión de información** que puede ser representada como estructuras de datos similares a objetos JSON. Es a este tipo de datos a los que se refiere el término «documentos», pero tal vez sería mejor referirnos a datos semiestructurados.

A diferencia de las bases de datos relacionales, donde todas las filas de una tabla tienen unas columnas predefinidas, estos documentos son unidades autocontenidas de información con una **estructura interna flexible.** Es decir, en lugar de requerir una estructura rígida de antemano, las bases de datos orientadas a documentos permiten que cada documento tenga su propio conjunto de campos y valores, lo que las hace ideales para **manejar datos heterogéneos** de forma simple y con la posibilidad de evolucionar.

Un aspecto clave de las bases de datos orientadas a documentos es que los datos se **almacenan como instancias únicas** en la base de datos, en lugar de distribuirse

© Universidad Internacional de La Rioja (UNIR)

en varias tablas, como vimos en los RDBMS. Al mismo tiempo, permiten almacenar elementos relacionados dentro de la misma colección, aunque los campos que definen esos elementos no sean comunes. Este concepto de colección es el equivalente al de tabla en el modelo relacional, pero hace más hincapié en el hecho de la **posible heterogeneidad de los documentos almacenados.**

Por otro lado, la capacidad de almacenar información relacionada en un solo documento **reduce la necesidad de realizar múltiples consultas y uniones,** lo que hace más eficiente, tanto la lectura como la escritura de los datos, y simplifica la escalabilidad horizontal del sistema.

```
{
  nombre: "juan",
  edad: 25,
  tipo: "alumno",
  asignaturas: ["rdbms", "nosql"],
  tutor: {
    nombre: "Tomás",
    catedra: "datos"
  }
}
```

Figura 1. Ejemplo de documento. Fuente: elaboración propia.

Un ejemplo de este tipo de bases de datos es MongoDB, que utiliza documentos similares a objetos JSON para almacenar los datos. MongoDB es una **opción popular** para una amplia gama de aplicaciones, desde sitios web hasta aplicaciones móviles y análisis de *big data.*

Bases de datos orientadas a grafos

Las bases de datos orientadas a grafos representan otro tipo particular de bases de datos NoSQL que se distinguen por su capacidad de **almacenar información en forma de grafos.** En este contexto, un grafo se compone de dos elementos

© Universidad Internacional de La Rioja (UNIR)

fundamentales: nodos y relaciones. Cada nodo representa a una entidad del modelo. Mientras que las relaciones definen cómo se conectan estos nodos, estableciendo asociaciones entre ellos. Esto hace que, en una base de este tipo, tanto las entidades como las relaciones son **elementos de primera clase.**

La estructura de los datos se **asemeja a la forma en que la información se organiza naturalmente** en muchos escenarios del mundo real. Por ejemplo, una red social puede representarse como un grafo, donde los usuarios son los nodos y sus conexiones de amistad son las relaciones. Del mismo modo, una red de carreteras, un mapa de rutas aéreas o un árbol genealógico pueden representarse eficientemente como grafos.

Representar estos casos como grafos proporciona un **rendimiento superior** al que tendría si intentáramos representarlos como datos en una base de datos relacional o de otro tipo. Esto hace que el campo de aplicación de las bases de datos de grafos sea amplio y que hayan experimentado un crecimiento notable en popularidad.

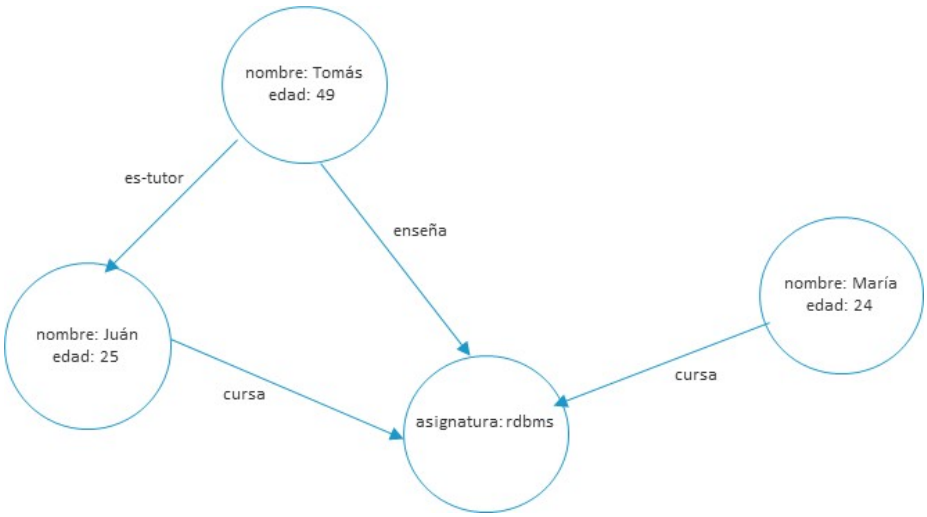


Figura 2. Ejemplo de datos en un grafo. Fuente: elaboración propia.

Un ejemplo de estas bases de datos es Neo4j. Esta se caracteriza por su **almacenamiento y procesamiento** de grafos de forma nativa.

© Universidad Internacional de La Rioja (UNIR)

El teorema CAP

Antes de profundizar en los ejemplos seleccionados de bases de datos NoSQL, es necesario que hagamos un breve aparte para hablar de un concepto importante para los sistemas distribuidos. El teorema CAP, también conocido como el teorema de Brewer, es particularmente relevante en el contexto de las bases de datos NoSQL. El teorema establece que es **imposible que un sistema distribuido garantice simultáneamente** las tres propiedades siguientes: **consistencia, disponibilidad y tolerancia** a particiones.

- **Consistencia *(Consistency)*.** Implica asegurar que todos los usuarios de la base de datos tengan la misma visión de estos en todo momento, independientemente del nodo al que estén accediendo. En otras palabras, ante la misma consulta, todos los usuarios recuperarán el mismo resultado.
- **Disponibilidad *(Availability)*.** El sistema garantiza que los usuarios podrán acceder a leer y modificar los datos, incluso si algunos de sus componentes fallan. Cada solicitud recibe una respuesta, ya sea de éxito o de fallo.
- **Tolerancia a particiones *(Partitioning)*.** Implica que el sistema continúe funcionando, incluso si se divide en subconjuntos aislados (particiones), debido a fallos de red.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

La limitación que implica el teorema CAP obliga a las bases de datos distribuidas a elegir solo dos de esas características en perjuicio de la tercera. Esto juega un papel crucial en la elección de la arquitectura que tendrá el sistema. A menudo, las bases de datos NoSQL **priorizan la disponibilidad y la tolerancia a particiones sobre la consistencia.** Por esta razón, se suele decir que tienen un modelo de consistencia eventual.

© Universidad Internacional de La Rioja (UNIR)

3.3. Modelo documental. MongoDB

Vamos a dar un vistazo más en profundidad a MongoDB como **ejemplo típico de una base de datos orientada a documentos.**

El desarrollo de MongoDB comenzó en 2007 por la compañía de *software* 10gen. La motivación principal era crear una base de datos que pudiera **abordar las necesidades de las aplicaciones web modernas,** que requerían una mayor flexibilidad y escalabilidad que las bases de datos relacionales tradicionales. Veamos algunas de las características que hacen de MongoDB una buena alternativa.

Una característica importante es que puede distribuirse fácilmente en múltiples servidores, lo que nos da un **buen rendimiento y tolerancia a fallos.** Al mismo tiempo, al utilizar un modelo documental, los datos no se almacenan en tablas con columnas predefinidas. En su lugar, MongoDB mantiene colecciones de documentos, conjuntos de documentos que desde el punto de vista de la aplicación representan entidades que pueden tener funciones similares, aunque sus atributos no lo sean. Esto es lo que permite una **mayor flexibilidad para agregar o modificar datos** sin necesidad de migraciones complejas.

Arquitectura

MongoDB se basa en una **arquitectura distribuida** que permite la escalabilidad horizontal y la alta disponibilidad. Los componentes principales de la arquitectura son dos procesos llamados mongod y mongos.

El **proceso principal** del servidor del sistema es el mongod. Este se encarga de almacenar los datos y procesar las diferentes operaciones de lectura y escritura.

Por otro lado, el proceso llamado mongos es el **encargado del enrutamiento** y actúa como un balanceador de carga y un punto de entrada para las consultas.

© Universidad Internacional de La Rioja (UNIR)

Para garantizar la durabilidad y la integridad de los datos, los procesos mongod se organizan en lo que se llama un **conjunto de réplicas.** Estos grupos de estos procesos mantienen copias sincronizadas de los mismos datos, de modo que se proporciona una mayor disponibilidad de los datos y una mejor tolerancia a fallos.

Por otro lado, el sistema permite la distribución y el escalado horizontal mediante una técnica llamada **fragmentación.** Esta consiste en dividir los datos en subconjuntos (fragmentos) y colocar cada uno de dichos fragmentos en un conjunto de replicas diferente.

En este punto es donde intervienen los procesos mongos, dado que son estos los que conocen qué fragmentos se encuentran en qué conjuntos de replicación y qué servidores, mongod, los componen. Por lo que puede **dirigir a los clientes al lugar adecuado para obtener o modificar** los datos necesarios.

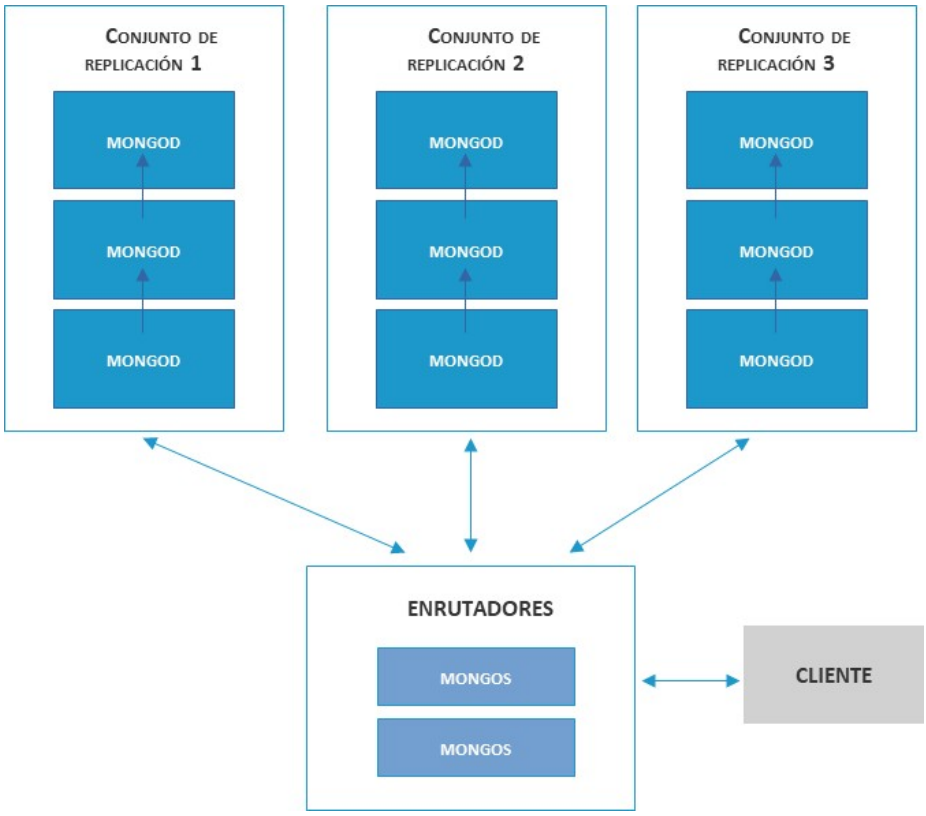


Figura 3. Esquema de arquitectura de MongoDB. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Con esta sencilla distribución en dos procesos con responsabilidades diferentes es como MongoDB consigue ser a la vez un **sistema escalable horizontalmente y robusto** en lo referente a la accesibilidad de los datos. En lo referente al teorema CAP, con esta arquitectura, MongoDB apuesta por las características AP en detrimento de C. Es decir, tiene alta disponibilidad y tolerancia a particiones, pero una **consistencia eventual, no estricta.**

Modelo de datos

Una vez vista la arquitectura, es hora de ver cómo MongoDB almacena los datos. Para ello, utiliza una estructura llamada documentos BSON, que son **estructuras con pares clave-valor,** muy similares a los objetos JSON, pero codificadas de forma binaria. Este enfoque ofrece una gran flexibilidad y se adapta bien a la naturaleza dinámica de los datos.

Como ya hemos comentado varias veces, estos BSON se **agrupan en colecciones,** que son conjuntos de documentos con funciones similares en la aplicación, aunque no necesariamente atributos iguales o similares.

```
{
  nombre: "juan",
  edad: 25,
  tipo: "alumno",
  asignaturas: ["rdbms", "nosql"],
  tutor: {
    nombre: "Tomás",
    catedra: "datos"
  }
}
```

Figura 4. Ejemplo de documento BSON. Fuente: elaboración propia.

Con esta estructura de datos, vamos a ser capaces de **realizar varias acciones mediante una API** que nos proporciona funciones muy variadas.

© Universidad Internacional de La Rioja (UNIR)

InsertOne nos permite **insertar un nuevo documento** en una *collection*.

```
db.personas.insertOne(
  {nombre: "Tomás", edad: 49, tipo: “profesor”, catedra: “datos” }
);
```

insertMany para **insertar múltiples documentos** en una *collection.*

```
db.personas.insertMany([
  {nombre: "Luís", edad: 25, tipo: “alumno”},
  {nombre: "María", edad: 23, tipo: “alumno”, matricula: true},
]);
```

find sirve para **buscar documentos** que coincidan con unos criterios dados.

```
// Encuentra todos las personas
db.personas.find();
// Encuentra las personas menores de 30 años
db.personas.find({edad: {$lt: 30}});
```

findOne, igual que el anterior, pero **busca un solo documento,** el primero, que coincida con los criterios.

```
// Encuentra la primera persona con nombre "Tomás"
db.personas.findOne({nombre: "Tomás"});
```

updateOne sirve para actualizar un solo documento que coincida con unos criterios.

```
// Cambia la edad de la primera persona con nombre "Tomás"
```


© Universidad Internacional de La Rioja (UNIR)

```
db.personas.updateOne(
  {nombre: "Tomás"},
  {$set: {edad: 50}}
);
```

updateMany **actualizará todos los documentos** que coincidan con criterios dados.

```
// Actualiza el tipo de todos las personas con edad mayor a 30
db.personas.updateMany(
  {edad: {$gt: 30}},
  {$set: {tipo: "retirado"}}
);
```

deleteOne **elimina un solo documento,** el primero que coincida los criterios.

```
// Elimina la persona con nombre "María"
db.personas.deleteOne({nombre: "María"});
```

deleteMany **eliminará todos los documentos** que coincidan los criterios dados.

```
// Elimina todos las personas con matricula
db.productos.deleteMany({matricula: true});
```

La API cubre todos los casos básicos de escritura y lectura de datos. Para profundizar más en las operaciones que permite MongoDB, puedes consultar un recurso en A Fondo.

© Universidad Internacional de La Rioja (UNIR)

3.4. Modelo columnar. Hbase

El ejemplo de base de datos columnar que vamos a tratar es Apache HBase. En realidad, HBase pertenece al conjunto de bases de datos orientados a familias de columnas. Es un sistema distribuido que tiene la **particularidad de trabajar sobre el sistema de archivos HDFS.** HBase está diseñado para proporcionar acceso a grandes cantidades de datos (miles de millones de filas y millones de columnas). Al usar HDFS como su sistema de almacenamiento, hereda de este el uso de *hardware commodity* para su funcionamiento.

La historia de HBase está estrechamente ligada a la publicación del artículo de BigTable por Google en 2006. Inspirado por el diseño de BigTable, HBase fue desarrollado inicialmente como parte del proyecto de búsqueda web Nutch de Doug Cutting, el creador de Hadoop. Con el crecimiento de Hadoop, HBase se convirtió en un **subproyecto de Hadoop** y, finalmente, se graduó como un proyecto de alto nivel de Apache en 2010.

La motivación detrás del desarrollo de HBase reside en la necesidad de una base de datos que pueda escalar horizontalmente, proporcionar acceso aleatorio rápido a los datos y, al mismo tiempo, asegurar una alta disponibilidad. Como vemos, características comunes a todas las bases de datos NoSQL, pero al mismo tiempo mediante una **aproximación diferente**, tanto en la arquitectura como en el modelo de datos.

Arquitectura

La arquitectura de HBase se compone de **tres componentes principales,** un servidor maestro (HMaster), uno o varios servidores de datos (RegionServer) y un mecanismo de coordinación (Zookeeper).

Los RegionServers son responsables de servir los datos a los clientes. Cada servidor

© Universidad Internacional de La Rioja (UNIR)

de regiones maneja un **conjunto de regiones, que son subdivisiones de los datos** en las tablas Hbase. Una región es una división **tanto horizontal como vertical** de los datos de una tabla. Horizontal, porque contiene solo una parte de las filas de la tabla, y vertical, porque contiene solo los datos de una familia de columnas. Cada RegionServer gestiona, como hemos dicho, varias de estas regiones de forma independiente, por lo que los datos de una tabla se encuentran distribuidos de múltiples formas entre ellos.

El HMaster, por su lado, es responsable de organizar las actividades en el clúster. Se encarga de asignar las regiones a los RegionServers y de la gestión, creación y eliminación de las tablas. Otra responsabilidad de HMaster es **equilibrar la carga de trabajo y almacenamiento** entre los distintos RegionServers.

Por último, ZooKeeper es un **servicio de coordinación distribuido** y, aunque no es parte del propio Hbase, dado que es un sistema genérico usado por otros sistemas además de este, juega un papel importante. Zookeeper se utiliza para **mantener la consistencia** y permitir la comunicación entre los RegionServers y el HMaster. También mantiene un seguimiento del estado de los diferentes procesos y de las regiones y proporciona mecanismos de bloqueo y sincronización.

© Universidad Internacional de La Rioja (UNIR)

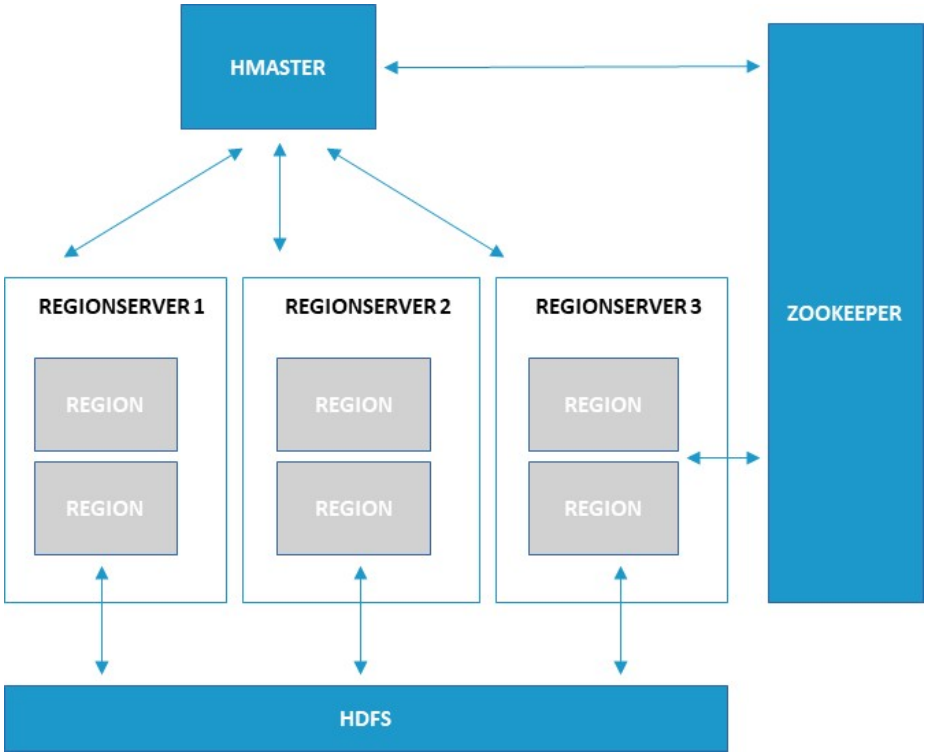


Figura 5. Arquitectura de Apache Hbase. Fuente: elaboración propia.

Modelo de datos

Los datos en HBase se organizan en tablas, que se componen de filas y columnas, similar a como se hace en el modelo relacional. Pero, a diferencia de este, las **filas se identifican mediante claves únicas siempre** y esta es la única forma eficiente de acceder a los datos. Por otro lado, las columnas se agrupan en familias que son unidades lógicas que agrupan columnas con relación funcional entre ellas y que nos puede interesar modificar o leer juntas. Como comentamos cuando hablamos de las regiones, esto permite la división tanto vertical como horizontal de las tablas a la hora de almacenarlas. Cuando una **región crece demasiado, se divide en regiones más pequeñas,** lo que permite a HBase mantener la escalabilidad horizontal.

La gran diferencia entre HBase y una base de datos relacional proviene de que no solo es posible hacer consultas sobre las filas almacenadas, a través de sus claves. También es posible realizar **consultas sobre los nombres de las columnas**

© Universidad Internacional de La Rioja (UNIR)

presentes en una determinada fila. Esto es así porque, si bien el número y nombre de las familias de columnas está fijado al crear la tabla, las columnas que contienen ni en nombre ni en cantidad lo están. Cada fila puede tener un conjunto totalmente diferente de columnas que el resto de las filas de la misma tabla. Dado que los nombres de las columnas y familias se almacenan junto a sus valores, es habitual que se utilicen nombres de columnas y familias lo más cortos posible para mejorar la capacidad de almacenamiento y la velocidad de lectura.

Para ilustrar esto, vamos a suponer que tenemos una aplicación de gestión de cursos, donde almacenamos los alumnos, profesores y asignaturas. En este ejemplo, se crearía una tabla llamada cursos, con una familia de columnas llamada datos, una familia llamada profesores y otra llamada alumnos. Cada fila representa un curso, y cada familia de columnas almacena **valores correspondientes** de dicho curso.

En esta tabla, la clave podría ser el nombre del curso, incluyendo su fecha de inicio. Mientras que la familia de columnas datos podría contener las columnas inicio, fin, estado, aula. Es posible que algunos cursos tuvieran más información en esta familia de columnas representada usando columnas diferentes. La familia de columnas profesores podría contener una columna por cada profesor de ese curso, el nombre de la columna podría ser el número de empleado del profesor y su valor indicar la asignatura que imparte en el curso. Esto permite **mantener en cada curso la lista completa** de los profesores.

Por otro lado, la familia de columnas alumnos, de forma similar, podría tener una columna por cada alumno cuyo nombre fuera el número su carné de alumno. El valor de esta columna sería, por ejemplo, una concatenación de sus notas en cada asignatura separadas por comas. Las familias, columnas y valores **dependerán mucho de cuál sea el objeto de la aplicación que usa estos datos.** Lo mismo ocurre con los valores de las claves, que deben ser únicos y adecuados para las búsquedas que necesite hacer la aplicación.

© Universidad Internacional de La Rioja (UNIR)

Clave de fila	Familia de columnas	Columna	Valor
bigdata:2023-09-01	datos	inicio	2024-09-01
		fin	2024-12-31
		estado	activo
		aula	A101
	profesores	2345	NoSQL
		7890	HDFS
	alumnos	20230001	8,P
		20230002	7,P
bigdata:2023-10-15	datos	inicio	2024-10-15
		fin	2025-03-15
		estado	Planificado
		aula	B202
	profesores	8765	NoSQL
		4321	HDFS
	alumnos	20230003	pendiente
curso-avanzado:2024-01-20	datos	inicio	2024-01-20
		fin	2024-06-20
		estado	Terminado
		aula	C303
	profesores	13579	Ingeniería de Software
	alumnos	20230004	9

Tabla 3. Tabla de cursos. Fuente: elaboración propia.

Este modelo es uno muy simple, es posible que aplicaciones más complejas dentro de este mismo ámbito funcional **requirieran un modelo mucho más complejo.**

A la hora de construir estos modelos, es necesario tener en cuenta que Hbase solo puede realizar **dos tipos de búsquedas,** por **clave exacta** o por **rango de claves.** Las búsquedas por clave exacta son las más eficientes, dado que solo necesitan localizar una fila concreta. Por otro lado, las búsquedas por rangos de claves se

© Universidad Internacional de La Rioja (UNIR)

apoyan en que HBase mantiene las filas ordenadas por los valores de las claves. Esto implica que las claves que empiezan por los mismos valores están juntas.

En nuestro ejemplo, vemos que las dos filas referentes al curso bigdata están consecutivas y ordenadas por el resto de su valor, en nuestro caso, su fecha de inicio. Esto quiere decir que no solo podemos obtener los datos de una instancia concreta de este curso, además podemos obtener los datos de instancias consecutivas en el tiempo. Podríamos buscar todas las veces que se ha dado el curso de Big Data entre enero y julio de 2023, buscando en el rango de claves entre bigdata:2023-01-01 y bigdata:2023-07-31. Todas esas filas estarían juntas en el disco.

Debemos hacer notar en este punto que, al estar la información almacenada en HDFS, HBase se **aprovecha de los mecanismos de replicación e integridad de los datos** que este sistema de ficheros proporciona. Del mismo modo, estas lecturas de datos consecutivos están también optimizadas por los propios mecanismos de lectura del HDFS, por lo que tiene sentido que este tipo de consultas sean las más habituales en modelos de datos implementados en esta base de datos.

Del mismo modo que vimos en MongoDB, vamos a presentar de forma breve la interacción con HBase y las posibles operaciones que podemos realizar. Como veremos, **estas operaciones son más simples en sus capacidades** que las que nos permitía esa otra base de datos.

Una vez conectados a Hbase, mediante su línea de comandos, podemos proceder a crear la tabla cursos, indicando las **tres familias de columnas** que queremos que contenga.

```
create 'cursos', 'datos', 'profesores', 'alumnos'
```

Para insertar registros en esta tabla, deberemos hacer **uso del comando put** de la siguiente forma:

© Universidad Internacional de La Rioja (UNIR)

```
put 'cursos', 'bigdata-2023-09-01', 'datos:inicio', '2023-09-01'

put 'cursos', 'bigdata-2023-09-01', 'datos:fin', '2023-12-31'

put 'cursos', 'bigdata-2023-09-01', 'datos:estado', 'activo'

put 'cursos', 'bigdata-2023-09-01', 'datos:aula', 'A101'

put 'cursos', 'bigdata-2023-09-01', 'profesores:2345', NoSQL'

put 'cursos', 'bigdata-2023-09-01', 'profesores:7890', 'HDFS'

put 'cursos', 'bigdata-2023-09-01', 'alumnos:20230001', '8,7,9'

put 'cursos', 'bigdata-2023-09-01', 'alumnos:20230002', '7,8,8'
```

Como podemos ver en el ejemplo, para **introducir todos los datos de una fila,** hemos tenido que hacer un put por cada una de sus columnas, indicando cada vez la tabla de destino y la clave de la fila por introducir. Por otro lado, vemos que las columnas son referenciadas mediante una concatenación del nombre de la familia y el nombre de la columna separados por el carácter «:». Por último, indicamos el valor que almacenar en dicha columna.

Podemos **obtener una fila por el valor** de su clave con el comando get.

```
// Obtenemos toda la fila por la clave

get 'cursos', 'bigdata:2023-09-01'

// Podemos limitar el resultado a una columna específica

get 'cursos', 'Curso1-2023-09-01', 'datos:estado'
```

Para **obtener los datos almacenados en un rango** de claves, podemos usar el comando scan.

```
// Si solo le damos el nombre de la tabla nos devolverá todos
```


© Universidad Internacional de La Rioja (UNIR)

```
// datos en dicha tabla

scan 'cursos'

// Podemos pedir que nos devuelva un rango de filas por clave

scan 'cursos', {

  STARTROW => 'bigdata:2023-01-01',

  ENDROW => 'bigdata:2023-07-31'

}
```

Como vemos, estas operaciones son muy sencillas y **no permiten búsquedas complejas,** pero, con un buen modelado de las claves y las columnas, seremos capaces de trabajar con miles de millones de filas a una velocidad que no es alcanzable por ninguna otra base de datos.

© Universidad Internacional de La Rioja (UNIR)

3.5. Casos de uso

Vamos a presentar dos casos de uso, uno con MongoDB y otro con Hbase, en los que se utilizan los puntos fuertes de estas bases de datos.

Comercio electrónico

En el primer caso, nos centraremos en una aplicación de comercio electrónico, específicamente, en el manejo de catálogos de productos. Las empresas de comercio electrónico, a menudo, necesitan manejar una gran cantidad de productos con atributos diversos. Algunos productos pueden tener **atributos específicos que otros no.** Por ejemplo, una camisa puede tener atributos, como talla, color y material, mientras que un libro puede tener atributos como autor, editorial y número de páginas.

Una base de datos relacional requeriría una tabla con un **gran número de columnas para acomodar todos los posibles atributos,** lo que puede llevar a una gran cantidad de campos vacíos y una estructura compleja de manejar.

En este caso, MongoDB, al ser una base de datos orientada a documentos, permite **almacenar cada producto como un documento con sus propios atributos.** Esto permite una mayor flexibilidad, al cubrir los posibles atributos de cualquier producto presente o futuro y evitar campos vacíos.

Un solo documento puede **contener todos los atributos de un producto,** incluyendo arrays para las tallas y los colores, si aplica a ese tipo de producto:

```
{
  "id": "123",
  "nombre": "Camiseta deportiva",
```


© Universidad Internacional de La Rioja (UNIR)

```
  "precio": 29.99,

  "tallas": ["S", "M", "L", "XL"],

  "colores": ["Rojo", "Azul", "Verde"],

  "material": "Algodón",

  "descripcion": "Camiseta deportiva de alta calidad..."

}
```

Además, al ser altamente escalable, lo que es crucial para las empresas de comercio electrónico que experimentan picos de tráfico durante eventos especiales o temporadas de compras, esta **elección de base de datos se ajusta muy bien** al caso.

Datos de IoT

El internet de las cosas (IoT) implica la **conexión de una gran cantidad de dispositivos,** como sensores, que generan un flujo constante de datos. HBase puede ser una solución muy eficaz para gestionar estos datos, especialmente, en escenarios donde se requiere escalabilidad, rendimiento y la capacidad de manejar datos en tiempo real.

Consideremos un sistema de monitoreo de una planta de energía que utiliza miles de sensores para recopilar datos sobre temperatura, presión, vibración y otros parámetros. Estos sensores transmiten datos continuamente, generando un **gran volumen de información que debe ser almacenada y analizada** para garantizar el funcionamiento seguro y eficiente de la planta. HBase puede escalar horizontalmente para manejar este gran volumen de datos, lo que permite a la planta de energía añadir más sensores sin afectar el rendimiento del sistema.

Otro punto para tener en cuenta es que HBase ofrece un alto rendimiento para

© Universidad Internacional de La Rioja (UNIR)

operaciones de escritura si los datos son secuenciales, tal como suele ser el flujo constante de datos de los sensores en tiempo real. Con una adecuada selección de las claves, podemos garantizar que los **datos se inserten siempre al final de las regiones.** Recordemos que HBase usa HDFS como almacenamiento y que este patrón de escritura es para el que este *filesystem* está optimizado. También nos proporcionará un buen rendimiento para operaciones de lectura de datos secuenciales, lo que nos permitirá realizar **analíticas sobre los datos históricos de manera rápida.**

Una base de datos relacional podría tener **dificultades para manejar la velocidad y el volumen de los datos generados** por los sensores. El esquema rígido de estas bases de datos podría dificultar la adición de nuevos tipos de sensores. HBase, con su flexibilidad de esquema y su capacidad de escalado, se adapta mejor a la naturaleza dinámica del IoT.

Clave de fila	Familia de columnas	Columna	Valor
sensor-0001-202312281000	datos	temperatura	85.5
		Presión	10.2
sensor-0002-202312281000	datos	vibración	0.8

Tabla 4. Ejemplo de uso de Hbase. Fuente: elaboración propia.

Este ejemplo muestra cómo se puede **utilizar HBase para almacenar datos** de diferentes sensores con diferentes *timestamps.*

Otra ventaja para tener en cuenta, en este caso, es la integración con el ecosistema de Hadoop, lo que permite ejecutar análisis a gran escala utilizando herramientas de este ecosistema para **obtener información valiosa** a partir de todos estos datos.

HBase se presenta, por tanto, como una **solución potente y flexible para la gestión de datos** de IoT, permitiendo escalabilidad, rendimiento y análisis eficiente de grandes volúmenes de datos en tiempo real.

© Universidad Internacional de La Rioja (UNIR)

Documentación oficial de MongoDB

MongoDB. (s. f.). *What is MongoDB?* https://www.mongodb.com/docs/manual/

Este artículo es fundamental para comprender los orígenes y fundamentos de HDFS. Proporciona una visión detallada de la arquitectura, diseño y funcionamiento de GFS, que sirvió como base para el desarrollo de HDFS.

© Universidad Internacional de La Rioja (UNIR)

Documentación oficial de Apache HBASE

Apache Software Foundation. (s. f.). *Apache HBASE Reference Guide.* https://hbase.apache.org/book.html

Esta web es la guía de referencia oficial para Apache Hbase y cubre una amplia gama de temas, partiendo de una introducción básica, y llegando hasta temas más avanzados.

© Universidad Internacional de La Rioja (UNIR)